Light FM Radio — Mobile App: Full SDLC Sprint Plan

> Kotlin Multiplatform (Android + iOS) | Compose Multiplatform | AzuraCast Backend

1. App Overview & Source Analysis

What the Web App Does (User-Facing)

Screen / Feature	Description
Home	Hero section, live indicator, featured shows (3), featured articles (3), live stats, recently played, up next, gospel CTA
Live Player	Fixed bottom bar — play/pause, volume, now-playing track info, album art, LIVE badge, streamer name
Teachings (Shows)	Full grid of shows, category filter chips, schedule info section
Show Detail	Title, host, description, image, schedule, category
Testimonies (Articles/News)	Full grid of articles, category filter chips, newsletter subscribe section
Article Detail	Full article content, share sheet (Facebook, Twitter, WhatsApp, Telegram, Email, Copy Link)
Respond (Contact)	Contact form (name, email, subject, category dropdown, message), contact info, gospel message box, FAQ accordion
Navigation	Top navbar: Home, Teachings, Testimonies, Respond
Footer	Social links (Facebook, Twitter, Instagram, YouTube), ministry links, copyright

Data Sources

Source	Endpoint	Polling
AzuraCast Now Playing	`https://app.lightfmradio.org/api/nowplaying/lightfm`	Every 15 seconds
AzuraCast Stream	`https://app.lightfmradio.org/listen/lightfm/radio.mp3`	Continuous
Shows API	`/api/shows` (PostgreSQL via Prisma)	On demand
Articles API	`/api/articles` (PostgreSQL via Prisma)	On demand
Contact API	`/api/contact` (POST)	On submit

AzuraCast API Response Shape (Now Playing)

```
now_playing.song { title, artist, art, album }
now_playing.duration
now_playing.playlist
playing_next.song { title, artist, art, album }
playing_next.duration
playing_next.playlist
song_history[].{ played_at, song { title, artist, art, album }, playlist }
live.is_live
live.streamer_name
live.broadcast_start
listeners.current
listeners.unique
listeners.total
```

Data Models (from Prisma Schema)

Show

```
id, title, host, description, image, schedule, category, createdAt, updatedAt
```

Article

```
id, title, excerpt, content, date, category, featured, createdAt, updatedAt
```

---

2. Tech Stack (as specified)

Layer	Technology
UI Framework	Compose Multiplatform (KMP)
Navigation	Compose Navigation with Type Safety
Architecture	Circuit (Slack) — Presenter/UI/State pattern
Design System	Material 3 (M3)
Styling	Compose Modifiers
Icons	Material Symbols / Lucide-Kotlin
Image Loading	Coil
HTTP Client	Ktor Client
JSON	Kotlinx Serialization
Local Cache	Room (Android) / SQLDelight (shared)
Remote Data/Cache	Store5

Layer	Technology
DI	Koin
Concurrency	Kotlin Coroutines & Flow
Dependency Mgmt	Version Catalogs (`libs.versions.toml`)
Build	Gradle Kotlin DSL (`.gradle.kts`)
Audio Playback	ExoPlayer (Android) / AVPlayer (iOS via expect/actual)

---

3. Project Structure (Recommended)

```
lightfm-mobile/
├── androidApp/
│   └── src/main/
├── iosApp/
├── shared/
│   └── src/
│       ├── commonMain/
│       │   ├── data/
│       │   │   ├── api/          # Ktor clients
│       │   │   ├── db/           # SQLDelight schemas
│       │   │   ├── model/        # Data classes + serialization
│       │   │   └── repository/   # Store5 repositories
│       │   ├── domain/
│       │   │   └── usecase/
│       │   └── ui/
│       │       ├── home/
│       │       ├── player/
│       │       ├── shows/
│       │       ├── articles/
│       │       ├── contact/
│       │       └── common/
│       ├── androidMain/          # expect/actual Android
│       └── iosMain/              # expect/actual iOS
├── gradle/
│   └── libs.versions.toml
└── build.gradle.kts
```

---

4. Design Phase (Pre-Sprint)

Design Deliverables

- [ ] **Brand tokens** — Extract colors from web app (accent, accent-alt, background, primary, foreground, muted-foreground, border) into M3 `ColorScheme` (light + dark)
- [ ] **Typography** — Map web serif fonts to M3 `Typography` scale (Display, Headline, Title, Body, Label)
- [ ] **Component inventory** — Map each web component to its M3 equivalent (see table below)

- [ ] **Wireframes** — Low-fidelity screens for all 6 screens + persistent player bar
- [ ] **Hi-fi mockups** — Figma/equivalent for all screens in light and dark mode
- [ ] **Prototype** — Clickable flow: Home → Teachings → Show Detail, Home → Testimonies → Article Detail, Contact form

Web → M3 Component Mapping

Web Component	M3 Equivalent
Navbar	`NavigationBar` (bottom) + `TopAppBar`
Player (fixed bottom)	Persistent `BottomSheet` or custom `Scaffold` slot
ShowCard / NewsCard	`ElevatedCard`
Category filter chips	`FilterChip` row in `LazyRow`
Contact form inputs	`OutlinedTextField`
Category select dropdown	`ExposedDropdownMenuBox`
Submit button	`Button` (filled)
Live badge	`Badge` with red tint
Share sheet	`ModalBottomSheet` with share options
FAQ accordion	`ListItem` with expand/collapse
Live indicator (pulsing dot)	Custom `Canvas` animation
Volume slider	`Slider`
Toast / success state	`Snackbar`
Social icons	`IconButton` with Coil SVG or Material Symbols

---

5. Sprint Plan

Sprint 0 — Project Setup & Infrastructure (1 week)

Goal: Runnable skeleton with DI, navigation, and network layer wired up.

Tasks

- [ ] Create KMP project in Android Studio (Kotlin Multiplatform wizard)
- [ ] Configure `libs.versions.toml` with all library versions
- [ ] Set up `build.gradle.kts` for `shared`, `androidApp`, `iosApp` modules
- [ ] Add all dependencies: Ktor, Kotlinx Serialization, Coil, Koin, Circuit, Store5, Room/SQLDelight, ExoPlayer
- [ ] Configure Koin modules (network, repository, presenter)

- [ ] Set up Compose Navigation with type-safe routes:
  - `Route.Home`
  - `Route.Shows`
  - `Route.ShowDetail(id: String)`
  - `Route.Articles`
  - `Route.ArticleDetail(id: String)`
  - `Route.Contact`
- [ ] Create `AppScaffold` with `NavigationBar` (Home, Teachings, Testimonies, Respond)
- [ ] Create `LightFmTheme` with M3 `ColorScheme` matching web brand colors (light + dark)
- [ ] Create `Typography` scale matching web serif/sans-serif usage
- [ ] Set up CI (GitHub Actions): build + lint on PR

Definition of Done: App launches, bottom nav works, theme renders correctly on Android emulator.

Sprint 1 — AzuraCast Integration & Live Player (2 weeks)

Goal: Fully functional live radio player with real-time now-playing data.

Data Layer Tasks

- [ ] Define `NowPlayingResponse` data class with `@Serializable` matching AzuraCast API shape
- [ ] Create `AzuraCastApiService` (Ktor) — `GET /api/nowplaying/lightfm`
- [ ] Create `NowPlayingRepository` using Store5:
  - Fetcher: Ktor call
  - Source of truth: in-memory (no DB needed for live data)
  - Refresh interval: 15 seconds via `Flow` + `repeatOnLifecycle`
- [ ] Define domain models: `Track`, `LiveInfo`, `ListenerStats`, `SongHistoryItem`

Audio Layer Tasks

- [ ] Create `expect class RadioPlayer` in `commonMain`
- [ ] Implement `actual class RadioPlayer` in `androidMain` using ExoPlayer:
  - `play()` — loads stream URL and starts playback
  - `pause()`
  - `setVolume(Float)`
  - Auto-recovery on stream error/stall (3-second retry, matching web behavior)
- [ ] Implement `actual class RadioPlayer` in `iosMain` using AVPlayer
- [ ] Expose player state as `StateFlow<PlayerState>` (isPlaying, volume, error)

UI Tasks (Circuit Pattern)

- [ ] `PlayerPresenter` — subscribes to `NowPlayingRepository` + `RadioPlayer` state, emits `PlayerUiState`
- [ ] `PlayerUiState`:

```
data class PlayerUiState(
  val isPlaying: Boolean,
  val volume: Float,
  val currentTrack: Track?,
  val isLive: Boolean,
  val streamerName: String?,
  val playingNext: Track?,
  val songHistory: List<SongHistoryItem>,
  val listeners: ListenerStats?
)
```

- [ ] `PlayerBar` composable (persistent, docked above `NavigationBar`):
  - Album art (Coil `AsyncImage`, 48dp, rounded corners)
  - Track title (truncated, 1 line)
  - Artist / streamer name (truncated, 1 line)
  - LIVE badge (red, pulsing) when `isLive = true`
  - Play/Pause `IconButton` (M3 `FilledIconButton` with accent color)
  - Volume `Slider` (compact, 120dp wide)
  - Playlist label when not live
- [ ] Wire `PlayerBar` into `AppScaffold` bottom slot

QA Checklist — Sprint 1:

- [ ] Player starts/stops on button tap
- [ ] Now-playing info updates every 15 seconds without UI flicker
- [ ] LIVE badge appears/disappears correctly
- [ ] Stream auto-recovers after simulated network drop (airplane mode test)
- [ ] Volume slider adjusts audio in real time
- [ ] Album art loads and shows placeholder when null
- [ ] Player persists across screen navigation

Sprint 2 — Home Screen (1.5 weeks)

Goal: Full home screen matching web layout, with live data.

Data Layer Tasks

- [ ] Create `ShowsApiService` (Ktor) — `GET /api/shows`
- [ ] Create `ArticlesApiService` (Ktor) — `GET /api/articles`
- [ ] Create `ShowsRepository` (Store5) with Room/SQLDelight cache
- [ ] Create `ArticlesRepository` (Store5) with Room/SQLDelight cache
- [ ] Define `Show` and `Article` domain models

UI Tasks

- [ ] `HomePresenter` — fetches top 3 shows + top 3 articles, emits `HomeUiState`
- [ ] `HomeScreen` composable:
  - **Hero section**: Brand tagline ("Listen, Read & Witness"), subtitle, "Listen Now" `FilledButton`, "Accept Jesus" `OutlinedButton`, `LiveIndicator` composable
  - **LiveIndicator**: Animated pulsing red dot + "LIVE" text (Canvas animation)
  - **Featured Teachings**: `LazyRow` or 3-item vertical list of `ShowCard`
  - **Biblical Truth & Testimonies**: 3-item list of `ArticleCard`
  - **"View All" links**: Navigate to Shows / Articles screens
  - **Live on Air section**: `ListenerStatsCard`, `LiveActivityCard`, `UpNextCard`, `RecentlyPlayedCard`
  - **Gospel CTA section**: "Are You Ready for Jesus?" with two buttons → Contact screen
- [ ] `ShowCard` composable:
  - Coil image (16:9 aspect ratio)
  - Title, host, schedule, category `AssistChip`
- [ ] `ArticleCard` composable:
  - Title, excerpt (2 lines max), date, category `AssistChip`
  - "Read More" text link
- [ ] `ListenerStatsCard`: Current / Daily Reach / Monthly Impact counters
- [ ] `LiveActivityCard`: Rotating mock activity feed with slide animation
- [ ] `UpNextCard`: Next track art, title, artist
- [ ] `RecentlyPlayedCard`: Scrollable list of last 5 tracks with timestamps

QA Checklist — Sprint 2:

- [ ] Home loads shows and articles from API
- [ ] Skeleton loading state shown while fetching
- [ ] Empty state shown if API returns no data
- [ ] "Listen Now" button triggers player play
- [ ] "Accept Jesus" / "Respond" buttons navigate to Contact
- [ ] "View All Teachings" navigates to Shows screen
- [ ] "Read All Articles" navigates to Articles screen

- [ ] Listener stats update with each now-playing poll
- [ ] LiveActivity rotates every 6 seconds
- [ ] Screen is scrollable end-to-end without clipping

Sprint 3 — Teachings (Shows) Screen & Show Detail (1 week)

Goal: Full shows listing with category filtering and detail view.

UI Tasks

- [ ] `ShowsPresenter` — fetches all shows, manages selected category filter
- [ ] `ShowsScreen` composable:
  - Page header: "Christian Teachings & Programs" title + subtitle
  - Category filter: `LazyRow` of `FilterChip` (All + dynamic categories from data)
  - Shows grid: `LazyVerticalGrid` (2 columns on phone, 3 on tablet)
  - Empty state when filter returns no results
  - Schedule info card at bottom (Mon–Fri, Midweek, Saturday, Sunday slots)
- [ ] `ShowDetailScreen` composable:
  - Hero image (Coil, full width, 16:9)
  - Title (`HeadlineLarge`)
  - Host name with person icon
  - Category `AssistChip`
  - Schedule with clock icon
  - Description body text
  - Back navigation (`TopAppBar` with back arrow)

QA Checklist — Sprint 3:

- [ ] All shows load and display correctly
- [ ] Category filter chips filter the grid in real time
- [ ] "All" chip resets filter
- [ ] Tapping a show navigates to detail screen
- [ ] Detail screen back button returns to shows list
- [ ] Grid handles 1, 2, and many items without layout breaks
- [ ] Images load with placeholder and error fallback

Sprint 4 — Testimonies (Articles) Screen & Article Detail (1 week)

Goal: Full articles listing with category filtering, detail view, and share functionality.

UI Tasks

- [ ] `ArticlesPresenter` — fetches all articles, manages selected category filter
- [ ] `ArticlesScreen` composable:
  - Page header: "Biblical Truth & Testimonies"
  - Category filter: `LazyRow` of `FilterChip`
  - Articles list: `LazyColumn` of `ArticleCard`
  - Empty state
  - Newsletter subscribe section (email input + subscribe button — UI only, no backend in mobile v1)
- [ ] `ArticleDetailScreen` composable:
  - Category badge (`SuggestionChip`)
  - Title (`HeadlineLarge`, centered)
  - Date with clock icon
  - Excerpt (italic, accent left border — use `Box` with `drawBehind`)
  - Full content body (`Text` with `whitespace-pre-wrap` equivalent)
  - Share button → `ModalBottomSheet` with:
    - Facebook, Twitter, WhatsApp, Telegram, Email share intents (Android `Intent.ACTION_SEND`)
    - Copy link button with "Copied!" feedback (`Snackbar`)
  - Back navigation

QA Checklist — Sprint 4:

- [ ] Articles load and display correctly
- [ ] Category filter works
- [ ] Tapping article navigates to detail
- [ ] Full article content renders with correct formatting
- [ ] Share sheet opens on share button tap
- [ ] Each share option opens the correct app/intent
- [ ] Copy link copies URL and shows snackbar confirmation
- [ ] Back navigation works from detail

Sprint 5 — Contact / Respond Screen (1 week)

Goal: Fully functional contact form with validation and submission.

Data Layer Tasks

- [ ] Create `ContactApiService` (Ktor) — `POST /api/contact`
- [ ] Define `ContactRequest` data class: `name, email, subject, category, message`
- [ ] Define `ContactRepository` with submit function returning `Flow<Result<Unit>>`

UI Tasks

- [ ] `ContactPresenter` — manages form state, validation, submission
- [ ] `ContactScreen` composable:
  - Page header: "Your Spiritual Journey Matters"
  - Contact info section:
    - Email: `lcministries254@gmail.com` (tappable `mailto:` intent)
    - Phone: `0713710041` (tappable `tel:` intent, labeled "Prayer Line 24/7")
    - Gospel message box (John 3:16 summary + Romans 10:9 quote)
  - Contact form:
    - Name `OutlinedTextField`
    - Email `OutlinedTextField` (keyboard type: email)
    - Subject `OutlinedTextField`
    - Category `ExposedDropdownMenuBox` with options:
      - I Want to Accept Jesus Christ
      - I Need Prayer
      - I Need Spiritual Counseling
      - I Have Questions About Faith
      - I Want to Volunteer
      - I Want to Give/Support Ministry
      - Other
    - Message `OutlinedTextField` (minLines = 5)
    - Submit `Button` with loading state
    - Error `Text` on failure
  - Success state: checkmark animation + "Thank You, Friend!" message + John 14:27 quote (auto-resets after 5 seconds)
  - FAQ section: expandable `ListItem` items for 4 gospel questions
- [ ] Form validation: all fields required, email format check

QA Checklist — Sprint 5:

- [ ] All form fields accept input
- [ ] Email field shows keyboard with `@` key
- [ ] Category dropdown opens and selects correctly

- [ ] Submit with empty fields shows validation errors
- [ ] Submit with invalid email shows error
- [ ] Successful submit shows success state
- [ ] Success state auto-resets after 5 seconds
- [ ] API error shows error message
- [ ] Email address tap opens mail app
- [ ] Phone number tap opens dialer
- [ ] FAQ items expand and collapse

Sprint 6 — Polish, Accessibility & Performance (1 week)

Goal: Production-quality UX, accessibility compliance, and performance optimization.

Polish Tasks

- [ ] Add `AnimatedVisibility` / `animateContentSize` transitions between screens
- [ ] Add `shimmer` skeleton loading for all list screens (shows, articles, home sections)
- [ ] Add pull-to-refresh (`PullToRefreshBox`) on Shows and Articles screens
- [ ] Add `SwipeRefresh` on Home screen live stats section
- [ ] Implement dark mode — verify all colors render correctly in both themes
- [ ] Add app icon (use `Light-FM-Png-Logo.svg` / `Light-FM-Jpeg-Logo.png` from brand assets)
- [ ] Add splash screen (M3 `SplashScreen` API)
- [ ] Add social media links in footer/about section (Facebook, Twitter, Instagram, YouTube)

Accessibility Tasks

- [ ] Add `contentDescription` to all `IconButton` and `Image` composables
- [ ] Verify minimum touch target size (48dp) for all interactive elements
- [ ] Test with TalkBack (Android) — all interactive elements must be reachable and labeled
- [ ] Verify color contrast ratios meet WCAG AA (4.5:1 for text)
- [ ] Add `semantics { heading() }` to section titles
- [ ] Ensure `Slider` (volume) has accessible value description

Performance Tasks

- [ ] Verify Coil image caching is configured (disk + memory cache)
- [ ] Verify Store5 cache TTL is set appropriately (shows/articles: 5 minutes)
- [ ] Profile with Android Studio Profiler — no frame drops on scroll
- [ ] Verify no memory leaks from audio player (use LeakCanary)
- [ ] Verify background polling stops when app is backgrounded (use `Lifecycle.repeatOnLifecycle`)

Sprint 7 — Testing (STLC) (1.5 weeks)

Goal: Comprehensive test coverage across unit, integration, and UI layers.

Unit Tests (JUnit5 + Turbine + MockK)

- [ ] `NowPlayingRepository` — verify polling interval, data mapping, error handling
- [ ] `ShowsRepository` — verify cache hit/miss behavior, Store5 refresh logic
- [ ] `ArticlesRepository` — same as above
- [ ] `ContactRepository` — verify request serialization, success/error flows
- [ ] `HomePresenter` — verify `UiState` emissions for loading, success, error
- [ ] `PlayerPresenter` — verify state transitions (play, pause, track change, live toggle)
- [ ] `ShowsPresenter` — verify category filter logic
- [ ] `ArticlesPresenter` — verify category filter logic
- [ ] `ContactPresenter` — verify form validation rules, submission flow
- [ ] `RadioPlayer` (Android) — verify play/pause/volume/recovery behavior (Robolectric)

Integration Tests

- [ ] Ktor mock engine tests for all API services
- [ ] Room/SQLDelight migration tests
- [ ] Store5 cache + network integration tests

UI Tests (Compose Testing)

- [ ] `HomeScreen` — shows and articles render, navigation buttons work
- [ ] `PlayerBar` — play/pause toggle, LIVE badge visibility, track info display
- [ ] `ShowsScreen` — filter chips filter grid, empty state renders
- [ ] `ShowDetailScreen` — all fields render correctly
- [ ] `ArticlesScreen` — filter chips work, empty state renders
- [ ] `ArticleDetailScreen` — content renders, share sheet opens
- [ ] `ContactScreen` — form validation, success state, FAQ expand/collapse

Manual QA Test Cases

ID	Test Case	Expected Result
TC-01	Launch app with no internet	Cached content shown, error banner displayed
TC-02	Tap play, then switch to another app	Audio continues in background
TC-03	Receive phone call while playing	Audio pauses, resumes after call

ID	Test Case	Expected Result
TC-04	Stream drops mid-play	Auto-recovery within 3 seconds
TC-05	Submit contact form with all fields	Success state shown, form resets
TC-06	Submit contact form with empty name	Validation error on name field
TC-07	Tap share on article, select WhatsApp	WhatsApp opens with article title + URL
TC-08	Rotate device on any screen	Layout adapts, no data loss
TC-09	Navigate all screens via bottom nav	Correct screen shown, player persists
TC-10	Open article detail, press back	Returns to articles list at same scroll position
TC-11	Filter shows by category	Only matching shows displayed
TC-12	Filter articles by category	Only matching articles displayed
TC-13	Tap phone number on contact screen	Dialer opens with number pre-filled
TC-14	Tap email on contact screen	Mail app opens with address pre-filled
TC-15	Dark mode toggle	All screens render correctly in dark theme
TC-16	TalkBack navigation through player bar	All controls announced correctly
TC-17	Volume slider at 0	Audio muted, icon changes to mute
TC-18	App cold start	Splash screen shown, home loads within 3 seconds
TC-19	Pull to refresh on shows screen	Fresh data fetched from API
TC-20	Now-playing updates while on home screen	Track info updates without full reload

Sprint 8 — Release Preparation (1 week)

Goal: App ready for Play Store submission.

Tasks

- [ ] Set `applicationId`, `versionCode`, `versionName` in `build.gradle.kts`
- [ ] Configure ProGuard/R8 rules for Ktor, Kotlinx Serialization, Koin, Circuit
- [ ] Generate signed release APK / AAB
- [ ] Create Play Store listing:
  - App name: "Light FM Radio"
  - Short description (80 chars)
  - Full description
  - Screenshots (phone + tablet): Home, Player, Shows, Articles, Contact
  - Feature graphic (1024×500)
  - App icon (512×512)
- [ ] Set up Firebase Crashlytics for crash reporting
- [ ] Set up Firebase Analytics for basic usage tracking (screen views, play events)
- [ ] Internal testing track release
- [ ] Closed testing (beta) with 10–20 users
- [ ] Address beta feedback
- [ ] Production release

---

6. Environment Variables / Config

```kotlin
// shared/src/commonMain/kotlin/config/AppConfig.kt
object AppConfig {
    const val STREAM_URL = "https://app.lightfmradio.org/listen/lightfm/radio.mp3"
    const val AZURACAST_API_URL = "https://app.lightfmradio.org/api/nowplaying/lightfm"
    const val BASE_API_URL = "https://lightfmradio.org" // web app base for
shows/articles/contact
    const val NOW_PLAYING_POLL_INTERVAL_MS = 15_000L
}
```

---

7. Sprint Summary

Sprint	Focus	Duration	Key Deliverable
Design	Wireframes, mockups, brand tokens	1 week	Figma prototype
0	Project setup, DI, navigation, theme	1 week	Runnable skeleton
1	AzuraCast integration, live player	2 weeks	Working radio player

Sprint	Focus	Duration	Key Deliverable
2	Home screen	1.5 weeks	Full home screen
3	Shows screen + detail	1 week	Teachings flow
4	Articles screen + detail + share	1 week	Testimonies flow
5	Contact / Respond screen	1 week	Contact form
6	Polish, accessibility, performance	1 week	Production-quality UX
7	Testing (STLC)	1.5 weeks	Test coverage + QA sign-off
8	Release prep	1 week	Play Store submission
Total		**~12 weeks**	

---

8. Key Architecture Decisions

Why Circuit over ViewModel?
Circuit's `Presenter` / `UiState` / `UiEvent` pattern maps directly to how the web app uses React context + hooks. Each screen has a `Presenter` that owns business logic and emits immutable `UiState` — the composable is purely a rendering function. This makes testing trivial (test the presenter, not the UI).

Why Store5 over manual caching?
The web app uses `useEffect` + `fetch` with no caching. On mobile, network conditions are unreliable. Store5 provides a `Fetcher` (network) + `SourceOfTruth` (Room/SQLDelight) pattern that gives offline-first behavior automatically, matching what users expect from a native app.

Why SQLDelight over Room for shared cache?
SQLDelight generates type-safe Kotlin from SQL and works in `commonMain`, meaning the same cache logic runs on both Android and iOS. Room is Android-only. For the initial Android-only release, Room is acceptable and simpler — migrate to SQLDelight when iOS is targeted.

Audio on iOS
The `expect/actual` pattern isolates platform audio code. The `commonMain` `RadioPlayer` interface is identical; only the implementation differs. This keeps all business logic (auto-recovery, state management) in shared code.

---

9. Social Media Links (for Footer/About)

Platform	URL
Facebook	https://www.facebook.com/lightfmkenya
Twitter/X	https://x.com/LightFmKenya
Instagram	https://www.instagram.com/lightfmkenya/
YouTube	https://www.youtube.com/@lcmstudiosKe/videos

"I am the light of the world." — John 8:12